

# DepChain - Stage 1

Antero Morgado<sup>[119213]</sup>, David Cruz<sup>[89377]</sup>, and Guilherme Gonçalves<sup>[106762]</sup>

University of Lisbon, Higher Technical Institute, PT  
Highly Dependable Systems - G08

**Abstract.** This report presents the design and implementation of DepChain, a simplified permissioned blockchain system developed for the Highly Dependable Systems course. In this first stage, the focus is on establishing a robust consensus layer based on the Basic HotStuff algorithm. Operating in a Byzantine environment over an unreliable UDP-based network, the system ensures dependability through a layered architecture of distributed abstractions, including authenticated perfect links and timeout-based failure detectors.

Developed in Java, the system leverages a Public Key Infrastructure (PKI) and cryptographic primitives to maintain a secure, append-only ledger among a static set of members. The design bridges the gap between client-side requests and BFT consensus primitives, ensuring that arbitrary node behavior and network faults, such as message corruption or duplication, are mitigated. Experimental results and a suite of intrusive tests demonstrate the system's correctness and its ability to maintain safety and liveness under adversarial conditions, providing a foundation for transaction processing layers.

**Keywords:** Distributed Systems · Blockchain · Byzantine.

## 1 System Design and Architecture

The DepChain system is designed as a distributed ledger maintained by a static set of  $n$  blockchain members, where up to  $f$  members can exhibit Byzantine (arbitrary) behavior. The architecture follows a client-server model where a library, linked to the application, acts as a gateway to the blockchain service. To ensure high dependability, the system is structured into modular layers, abstracting network unreliability and providing strong consensus guarantees.

### 1.1 Layered Architecture

The design leverages a hierarchy of distributed abstractions to bridge the gap between raw UDP communication and the high-level BFT consensus. This modularity allows for clear separation of concerns and easier testing of individual dependability properties.

1. **Communication Layer (UDP):** Given the prohibition of TLS/TCP, this layer manages raw datagrams. It is designed to approximate Fair Loss Links,

handling the reality of message drops, delays, and corruption without relying on built-in transport-level reliability.

2. **Authenticated Perfect Links:** Built on top of UDP, this layer provides authenticated and reliable communication between members.

*Integrity and authenticity* are ensured using digital signatures. Every outgoing message is serialized and signed using the sender's private key. Upon reception, the receiver verifies the signature using the sender's public key obtained from the system membership configuration. Messages whose signatures fail verification are discarded, preventing impersonation or message tampering.

To guarantee *no-duplication*, each message carries a unique identifier composed of the sender identifier and a locally generated message identifier. Replicas maintain a set of previously delivered message identifiers and discard any message that has already been processed.

Finally, *reliable delivery* is approximated through an acknowledgment and retransmission mechanism. When a message is received, the receiver sends an ACK message back to the sender. The sender stores pending messages and periodically retransmits them if no acknowledgment is received within a timeout interval. Once the corresponding acknowledgment is received, the message is removed from the pending buffer.

3. **Consensus Layer (Basic HotStuff):** This layer implements the Basic HotStuff BFT consensus protocol. The protocol proceeds through four phases: (Prepare, Pre-commit, Commit, and Decide). During these phases, replicas exchange votes that are aggregated into Quorum Certificates (QCs), which prove that a sufficient number of nodes have agreed on a proposal. Threshold signatures are used to efficiently validate the votes collected in each phase. Once a valid Commit Quorum Certificate (QC) is obtained, the proposal is decided and delivered to the upper layer. The protocol resists Byzantine behaviour such as delayed, missing or conflicting votes, ensuring that only proposals supported by a quorum of honest replicas can be committed, providing both safety and liveness guarantees.

4. **Application Layer:** The application layer represents the blockchain replica logic, coordinating client interactions, proposing commands to the consensus layer, and maintaining the blockchain state.

Each replica maintains an internal request registry, implemented as the `pendingClientRequests` table, which tracks the execution state of incoming client commands (PENDING or COMPLETED) and ensures that each command is processed exactly once, ignoring duplicates. Commands are identified by a unique key composed of the Client Id and the client Message Id. In addition to this state table, replicas maintain an ordered buffer of active requests, implemented as the `activeRequestsBuffer` structure. This buffer stores client requests that have been received but not yet committed, including those that arrive while a previous command is already in the consensus

pipeline, and preserves their arrival order. It functions as a request queue and allows replicas to re-propose pending commands in the event of a leader crash. When a replica becomes the leader of a new view, it scans this buffer and selects the first pending command to propose through the consensus protocol. This mechanism guarantees that client requests are not lost during view changes and preserves liveness even if a leader fails.

Communication between replicas is performed using a set of typed protocol messages corresponding to the phases of the Basic HotStuff algorithm. These include messages for view changes (`NEW_VIEW`), phase messages broadcast by the leader (`PREPARE`, `PRE_COMMIT`, `COMMIT`, and `DECIDE`), and vote messages sent by replicas (`PREPARE_VOTE`, `PRE_COMMIT_VOTE`, and `COMMIT_VOTE`). Client commands are encapsulated in `APPEND_STRING` messages, which contain a JSON payload specifying the client identifier, message identifier, and the string to append to the blockchain. Once the consensus protocol reaches a decision, the consensus layer notifies the replica of the decided command. The replica then executes the committed command, by appending the corresponding value to the local blockchain state and replying to the client confirming if the command has been successfully committed.

5. **Client Library:** The client is a simple terminal application that allows users to send commands to be executed by the blockchain replicas. In this first stage, the commands are limited to appending strings to the replicas' ledger. After broadcasting a command, the client waits for responses from  $n - f$  replicas to confirm that the operation was successfully committed.

## 1.2 Design Choices

### 1. Pacemaker Implementation

The Basic HotStuff protocol assumes the existence of a pacemaker mechanism responsible for advancing the view when the current leader fails to make progress. In our implementation, the pacemaker is implemented using a timeout-based mechanism integrated into the replica process.

Each replica starts a timer whenever a client request is pending and the system is waiting for a proposal or progress in the consensus protocol. The timer is implemented using a scheduled task that triggers after a predefined timeout interval. If the timeout expires without a decision being reached, the replica assumes that the current leader may have failed or is unresponsive. The replica then advances to the next view by invoking the `advanceView()` operation in the consensus module.

If the replica becomes the new leader, it immediately scans its buffer of pending client requests and proposes the first pending command to the consensus protocol. This mechanism ensures that progress can resume even if the previous leader crashes, preserving the liveness property of the protocol. The pacemaker timer is stopped whenever a decision is reached and restarted when new requests remain pending. This avoids unnecessary view changes

while the system is making progress.

## 2. Threshold Signature Aggregation

HotStuff relies on quorum certificates constructed from votes of multiple replicas. To implement this efficiently, our system uses a threshold signature library (`threshsig`) that allows multiple signature shares to be combined into a single authenticator representing the approval of a quorum of replicas.

The threshold key shares are generated before the system starts and distributed to all replicas. Each replica stores its private key share and uses it to sign its votes during the consensus protocol using the `signShare` operation.

During the PREPARE, PRE\_COMMIT, and COMMIT phases, replicas sign their vote over a byte representation of the vote data, which includes the view number, the vote type, and the hash of the proposed node. These signature shares are sent to the leader as part of the vote messages.

The leader collects the signature shares and verifies them once it receives at least  $n - f$  votes. Since a Byzantine leader may receive votes referring to different payloads, the leader reconstructs the signed data from the received messages and verifies candidate byte representations. The verification procedure tries all combinations of  $n - f$  collected signature shares and checks whether a valid threshold signature can be verified.

Once a valid set of shares is found, the leader constructs the quorum certificate for that phase, which serves as proof that a quorum of replicas approved the proposal.